

ISG012-03 - Firewalls de Perímetro e Filtragem Avançada - Prática Lab

Objetivo: Implementar um firewall de perímetro completo usando Linux Mint como gateway entre rede interna simulada e DMZ

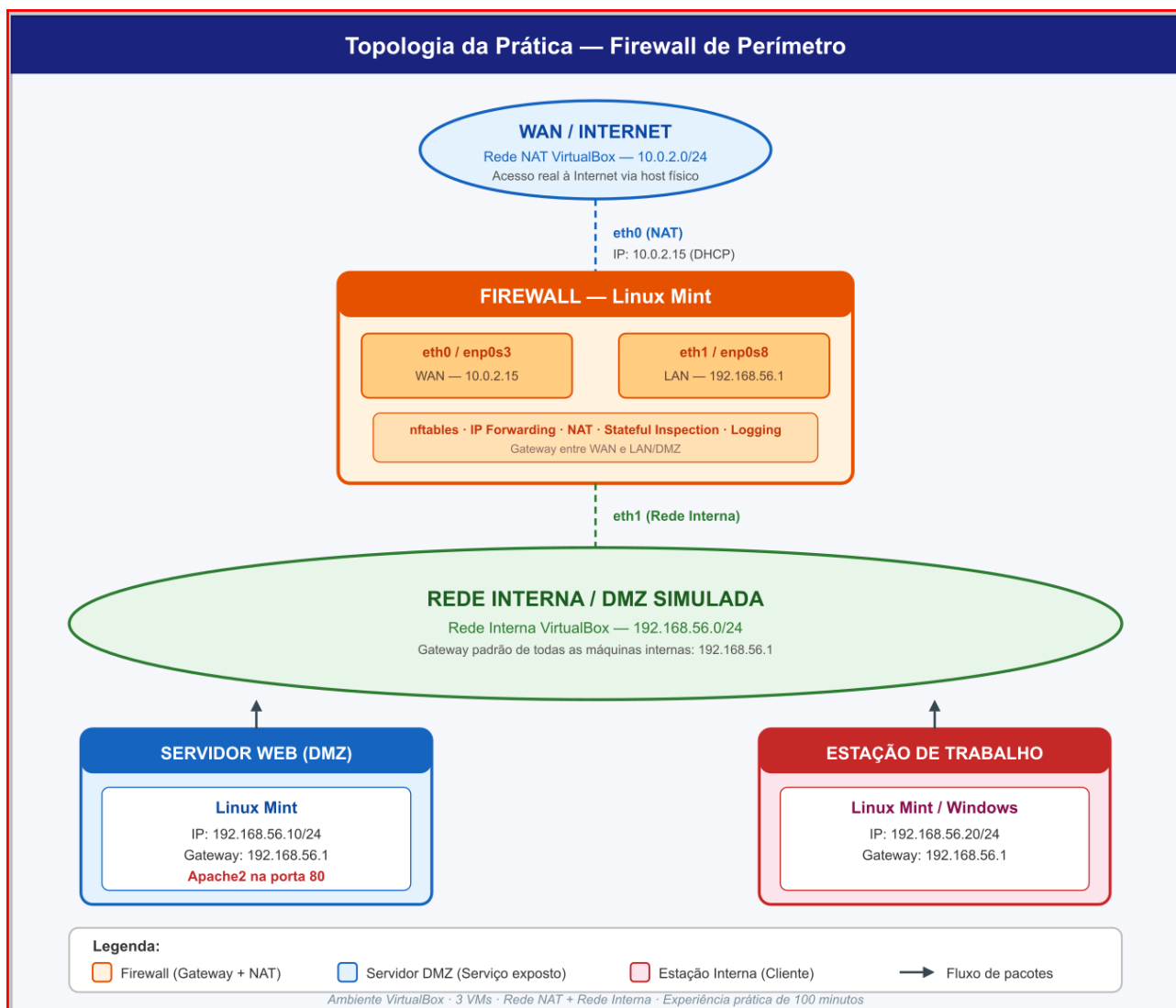
Ambiente final:

- 3 VMs no VirtualBox: Firewall (2 interfaces), Servidor DMZ (1 interface), Estação de Trabalho (1 interface)
- Ferramentas: nftables, bash, tcpdump, conntrack

Topologia da Experiência

Topologia final implementada:

Máquina	Interfaces	Rede	IP
Firewall	eth0 (NAT VirtualBox)	WAN simulada	10.0.2.15 (DHCP)
Firewall	eth1 (Rede Interna VBox)	LAN/DMZ	192.168.56.1/24
Servidor DMZ	eth0 (Rede Interna)	LAN	192.168.56.10/24
Estação	eth0 (Rede Interna)	LAN	192.168.56.20/24



Preparação do Ambiente (Parte 1)

Configuração final das VMs no VirtualBox:

1. **Criar 3 VMs** com Linux Mint (ou Windows sem admin para a estação)
2. **Adaptador 1** de todas as VMs: **Rede Interna** → nome intnet
3. **Firewall apenas** — Adaptador 2: NAT (simula Internet)
4. **Verificar interfaces** no firewall:
`ip link show`
 Resultado típico: enp0s3 (NAT) e enp0s8 (rede interna)
5. **Instalar nftables** em todas as VMs Linux:
`sudo apt update && sudo apt install nftables conntrack -y`
6. **Habilitar IP forwarding** (temporário):
`sudo sysctl -w net.ipv4.ip_forward=1`
Persistir editando /etc/sysctl.conf:
`sudo sed -i 's/#net.ipv4.ip_forward=1/net.ipv4.ip_forward=1/' /etc/sysctl.conf`

Configuração de Endereçamento IP

Resultado final da configuração de IPs estáticos:

No Firewall (interface interna):

```
sudo nmcli con mod "Wired connection 2" \  
  ipv4.addresses 192.168.56.1/24 \  
  ipv4.method manual \  
  connection.autoconnect yes  
sudo nmcli con down "Wired connection 2" && sudo nmcli con up "Wired connection 2"
```

No Servidor DMZ:

```
sudo nmcli con mod "Wired connection 1" \  
  ipv4.addresses 192.168.56.10/24 \  
  ipv4.gateway 192.168.56.1 \  
  ipv4.dns 8.8.8.8 \  
  ipv4.method manual  
sudo nmcli con down "Wired connection 1" && sudo nmcli con up "Wired connection 1"
```

Na Estação:

```
sudo nmcli con mod "Wired connection 1" \  
  ipv4.addresses 192.168.56.20/24 \  
  ipv4.gateway 192.168.56.1 \  
  ipv4.dns 8.8.8.8 \  
  ipv4.method manual  
sudo nmcli con down "Wired connection 1" && sudo nmcli con up "Wired connection 1"
```

Verificação final:

```
ip addr show  
ping 192.168.56.1    # do servidor/estação para o firewall
```

Script de Firewall: Estrutura Base

Estrutura final do script:

```
#!/bin/bash  
WAN_IF="enp0s3"  
LAN_IF="enp0s8"  
LAN_NET="192.168.56.0/24"  
DMZ_SERVER="192.168.56.10"  
  
# 1. Limpar regras  
# O ruleset é a coleção completa de todas as tabelas, chains e regras  
# que o nftables mantém ativas na memória do kernel para filtrar,  
# traduzir e registrar o tráfego de rede.  
nft flush ruleset  
  
# 2. Criar tabela filter (inet = IPv4+IPv6)  
nft add table inet filter  
  
# 3. Criar chains: input, forward, output (filter) + prerouting, postrouting (nat)
```

Chain INPUT (Filtragem Local)

Regras finais da chain INPUT:

```
nft add chain inet filter input { type filter hook input priority 0 \; policy drop \; }
```

Estado: aceitar conexões já estabelecidas

```
nft add rule inet filter input ct state established,related accept
```

Loopback

```
nft add rule inet filter input iifname "lo" accept
```

ICMP (ping) com rate limit

```
nft add rule inet filter input icmp type echo-request limit rate 5/second accept
```

```
nft add rule inet filter input icmpv6 type echo-request limit rate 5/second accept
```

SSH apenas pela interface interna

```
nft add rule inet filter input iifname "enp0s8" tcp dport 22 ct state new accept
```

Log e drop para tudo mais

```
nft add rule inet filter input log prefix "DROP-INPUT: " limit rate 5/minute drop
```

Política padrão: DROP — o firewall recusa tudo por padrão, exceto o explicitamente permitido.

Chain FORWARD (Filtragem entre Redes)

Regras finais da chain FORWARD:

```
nft add chain inet filter forward { type filter hook forward priority 0 \; policy drop \; }
```

Estado: conexões estabelecidas

```
nft add rule inet filter forward ct state established,related accept
```

LAN → Internet: DNS (TCP e UDP)

```
nft add rule inet filter forward iifname "enp0s8" oifname "enp0s3" ct state new tcp dport 53 accept
```

```
nft add rule inet filter forward iifname "enp0s8" oifname "enp0s3" ct state new udp dport 53 accept
```

LAN → Internet: HTTP e HTTPS

```
nft add rule inet filter forward iifname "enp0s8" oifname "enp0s3" ct state new tcp dport { 80, 443 } accept
```

LAN → Internet: ping limitado

```
nft add rule inet filter forward iifname "enp0s8" oifname "enp0s3" icmp type echo-request limit rate 5/second accept
```

```
# WAN → LAN: permitir tráfego DNAT (port forwarding) para porta 80
nft add rule inet filter forward iifname "enp0s3" oifname "enp0s8" tcp dport 80
ct state new accept
```

```
# Log de tentativas bloqueadas
nft add rule inet filter forward log prefix "DROP-FORWARD: " limit rate
5/minute
```

Conceito aplicado: Stateful Inspection — a rede interna confiável pode iniciar conexões para fora, mas o exterior não pode iniciar conexões para dentro (exceto via DNAT explícito).

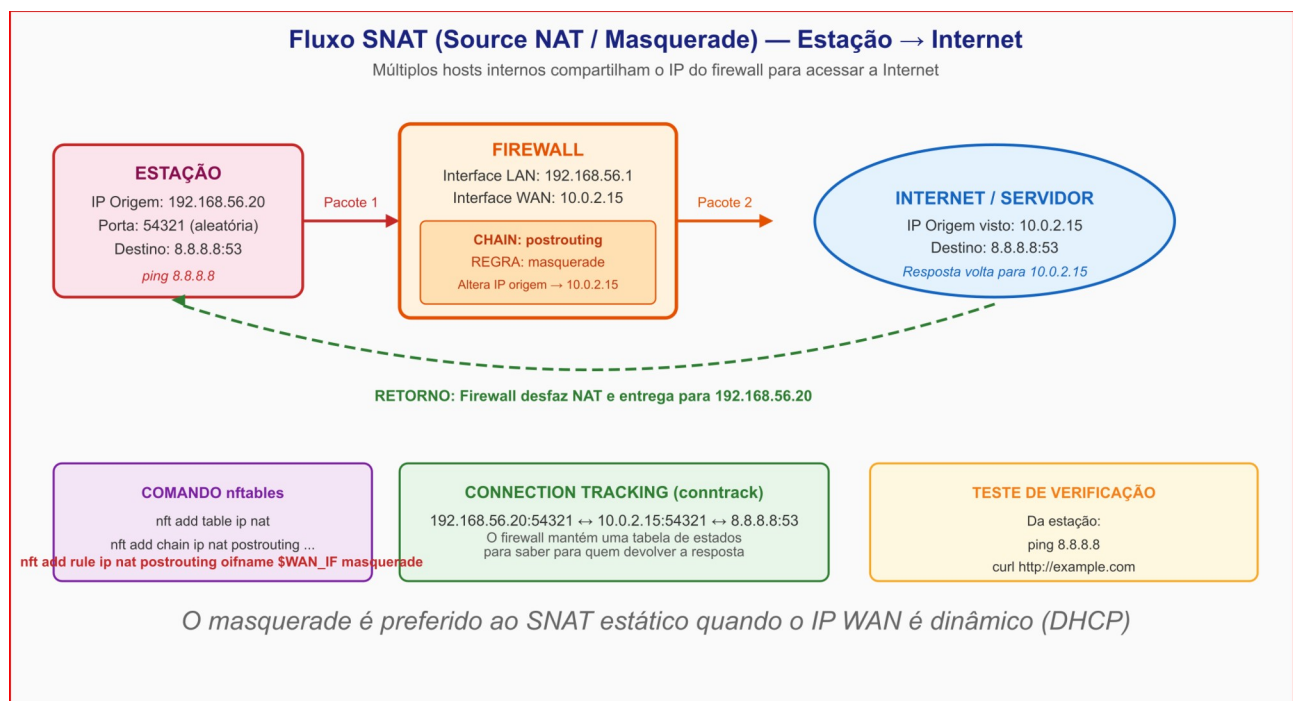
Tabela NAT e SNAT (Mascaramento)

Regras finais de NAT:

```
# Criar tabela NAT (IPv4 específico)
nft add table ip nat

nft add chain ip nat prerouting { type nat hook prerouting priority -100 \; }
nft add chain ip nat postrouting { type nat hook postrouting priority 100 \; }

# SNAT/Masquerade: rede interna compartilha IP do firewall
nft add rule ip nat postrouting oifname "enp0s3" masquerade
```



Teste final:

```
# Da estação (192.168.56.20):
ping 8.8.8.8
curl http://example.com
curl ifconfig.me # Deve retornar 10.0.2.15 (IP do firewall, não da estação)
```

DNAT (Port Forwarding)

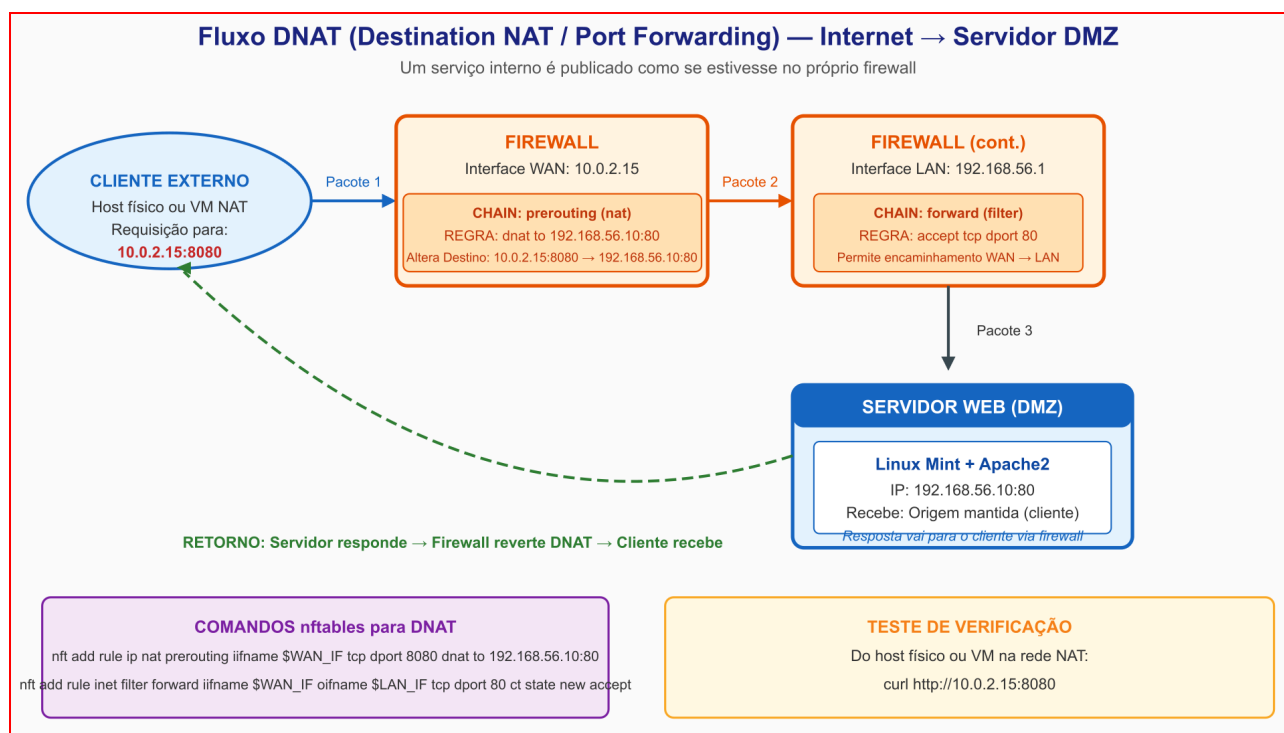
Regras finais de DNAT:

```
# Redireciona porta 8080 da WAN para servidor interno na porta 80
nft add rule ip nat prerouting iifname "enp0s3" tcp dport 8080 dnat to
192.168.56.10:80
```

```
# Regra de forward correspondente (já incluída no slide 29)
# nft add rule inet filter forward iifname "enp0s3" oifname "enp0s8" tcp dport
80 ct state new accept
```

Preparação do servidor DMZ:

```
sudo apt install apache2 -y
sudo systemctl enable apache2
sudo systemctl start apache2
sudo ss -tlnp | grep :80 # Verificar que está ouvindo
```



Teste final:

```
# Do host físico ou de uma VM na rede NAT (10.0.2.0/24):
curl http://10.0.2.15:8080
# Resultado: página padrão do Apache2 do servidor 192.168.56.10
```

Script Completo e Persistência

Este é o **produto final** da experiência — o script completo, pronto para execução:

Local sugerido: /home/aluno/firewall/setup_firewall.sh

Execução:

```
chmod +x /home/aluno/firewall/setup_firewall.sh
sudo /home/aluno/firewall/setup_firewall.sh
```

Persistência final (após execução bem-sucedida):

```
# Salvar ruleset atual
sudo nft list ruleset > /etc/nftables.conf
```

```
# Habilitar serviço no boot
sudo systemctl enable nftables
sudo systemctl start nftables
sudo systemctl status nftables -no-pager
```

Output final do nft list ruleset após execução:

```
aluno@firewall:~$ sudo nft list ruleset
```

```
table inet filter {
chain input {
type filter hook input priority filter; policy drop;
ct state established,related accept
iifname "lo" accept
icmp type echo-request limit rate 5/second accept
icmpv6 type echo-request limit rate 5/second accept
iifname "enp0s8" tcp dport 22 ct state new accept
log prefix "DROP-INPUT: " limit rate 5/minute drop
}

chain forward {
type filter hook forward priority filter; policy drop;
ct state established,related accept
iifname "enp0s8" oifname "enp0s3" ct state new tcp dport 53 accept
iifname "enp0s8" oifname "enp0s3" ct state new udp dport 53 accept
iifname "enp0s8" oifname "enp0s3" ct state new tcp dport { 80, 443 } accept
iifname "enp0s8" oifname "enp0s3" icmp type echo-request limit rate 5/second accept
iifname "enp0s3" oifname "enp0s8" tcp dport 80 ct state new accept
log prefix "DROP-FORWARD: " limit rate 5/minute
}

chain output {
type filter hook output priority filter; policy accept;
}
}

table ip nat {
chain prerouting {
type nat hook prerouting priority dstnat; policy accept;
iifname "enp0s3" tcp dport 8080 dnat to 192.168.56.10:80
}

chain postrouting {
type nat hook postrouting priority srcnat; policy accept;
oifname "enp0s3" masquerade
}
}
```

LEGENDA DE CORES

- Estrutura (tabelas, chains)
- Regras ACCEPT (permitidas)
- Regras DROP/LOG (bloqueadas)
- Políticas padrão (policy)

Testes e Verificação

Matriz de testes final — todos devem passar:

Matriz de Testes e Verificação do Firewall			
ORIGEM	COMANDO DE TESTE	RESULTADO ESPERADO	STATUS
Estação Interna 192.168.56.20	ping 8.8.8.8	Respostas ICMP recebidas NAT/SNAT funcionando	PASS
Estação Interna 192.168.56.20	curl http://example.com	HTML recebido com sucesso Forward HTTP/HTTPS OK	PASS
Estação Interna 192.168.56.20	curl ifconfig.me	Retorna 10.0.2.15 (IP do firewall) Confirma mascaramento SNAT	PASS
Host Físico / VM NAT Rede 10.0.2.0/24	curl http://10.0.2.15:8080	Página padrão do Apache2 DNAT: 8080 → 192.168.56.10:80	PASS
Host Externo Via interface WAN	ssh aluno@10.0.2.15	Connection timeout / refused SSH bloqueado na WAN (policy DROP)	BLOCK
Estação Interna Via interface LAN	ssh aluno@192.168.56.1	Login SSH bem-sucedido SSH permitido apenas na LAN	PASS
Firewall (local) Verificação de logs	sudo dmesg grep DROP	Linhas "DROP-INPUT" e "DROP-FORWARD" Tentativas bloqueadas registradas	PASS
Firewall (local) Estados de conexão	sudo conntrack -L head	Lista de conexões rastreadas Estados: ESTABLISHED, NEW, etc.	PASS
Firewall (local) Estatísticas de regras	sudo nft list ruleset -a	Contadores de pacotes por regra Verifica quais regras foram acionadas	PASS
Após reboot Teste de persistência	sudo nft list ruleset	Regras carregadas automaticamente Serviço nftables habilitado no boot	PASS
Todos os testes validam: NAT, DNAT, Stateful Inspection, Logging e Persistência VirtualBox + Linux Mint + nftables			

Comandos de verificação adicionais:

```
# Ver estados de conexão (conntrack)
sudo conntrack -L | head -20

# Ver contadores de pacotes por regra
sudo nft list ruleset -a

# Ver Logs de pacotes dropados
sudo dmesg | grep DROP
sudo journalctl -k | grep DROP
```

Troubleshooting Comum

Problema	Causa provável	Solução
nft: syntax error	Escape de ; incorreto	Usar \; em vez de ; nos comandos
NAT não funciona	IP forwarding desabilitado	cat /proc/sys/net/ipv4/ip_forward deve retornar 1
DNS não resolve	UDP 53 bloqueado	Verificar regra udp

Problema	Causa provável	Solução
		dport 53 accept na forward
DNAT não funciona	Sem regra de forward	Adicionar tcp dport 80 ct state new accept na forward
SSH pela WAN funciona	Regra input muito permissiva	Verificar iifname \$LAN_IF na regra SSH
Regras somem no reboot	Serviço nftables não habilitado	sudo systemctl enable nftables

Ferramentas de debug:

```

sudo tcpdump -i enp0s3 -n    # Capturar na WAN
sudo tcpdump -i enp0s8 -n    # Capturar na LAN
sudo nft trace                # Rastreamento detalhado nftables

```